



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 04 – Lecture 07: Splitting, Validation and Imbalance

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Splitting, cross-validation and imbalanced data

Topic focus: **a score is an estimate, and every estimate has a spread**

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.

One split
○○○○○○○

Strategies
○○○○○○○○○○

Cross-validation
○○○○○○○

Tuning
○○○○○

Imbalance
○○○○○○○○○○○○○○○○

Summary
○○

Quick Links

[One split](#)

[Split strategies](#)

[Cross-validation](#)

[Tuning honestly](#)

[Imbalanced data](#)

[Summary](#)

Agenda

A single train/test split

Split strategies: stratified, grouped, ordered

Cross-validation

Tuning honestly: validation sets and nested CV

Imbalanced data

Summary

How this lecture works

Every idea appears twice

1. **By hand** — twelve rows split three ways, a 1000-patient confusion matrix, two points and a straight line. Small enough to check on paper.
2. **In code** — the same arithmetic on real data, so you see that `StratifiedKFold` and `GroupKFold` do exactly that and nothing more.

Commit before you look

Five slides ask you to predict a number before it is revealed. Write your guess down. Being wrong on paper is how the idea sticks.

The centrepiece: **the same three lines of code report $F_1 = 0.894$ or $F_1 = 0.113$, depending on where you put one of them.**

Where we are

Last lecture chose the columns

Scaling, encoding, PCA, selection. It ended on one measurement: **selecting features before splitting scored 0.8475 on data made of coin flips.**

That was a symptom

The disease is the *split* — what goes in it, when it happens, and what it is allowed to know.

Today's four questions

How reliable is a single held-out score?
What if the rows are not independent?
How many folds? And what do I do when one class is 1% of the data?

One sentence to carry through

Anything you learn from the data — a mean, a category list, a feature subset, a hyper-parameter, a resampled row — is part of the model, and belongs inside the loop.

What a held-out score is actually estimating

The performance the model would have on *new rows from the same source*.

Why we hold anything out at all

A model can memorise. Training accuracy measures memory; held-out accuracy measures *generalisation*. Every number you ever report should come from rows the model has not seen.

But the held-out rows were also chosen at random

So the number you report is a **random variable**. It has a mean, and it has a spread, and **almost nobody reports the spread**.

`train_test_split(X, y, test_size=0.25, random_state=42)` — the last argument is not a formality. It is a coin toss with your result on it.

Four datasets, one seed — and why this deck has sweep

One arbitrary seed

`np.random.default_rng(0),`
`random_state=0`. Every listing that splits,
shuffles or resamples shows it.

Three data seeds, kept

The 99:1 problem, the 40-patient study and
the 600-day series are *defined* by their seeds,
so those keep their own names and values.

And a great many sweeps, on purpose

500 splits, 40 shuffles, 20 nested trials: these change seed every repetition, because **the spread across repetitions is the subject of this lecture**. Their seeds run `range(0, 500)`, `range(0, 40)`, so the sweep still reproduces exactly. Run the code as written and every number here comes back — except the seconds, which are labelled where they appear.

By hand: twelve rows, four held out

Twelve patients, nine healthy (0) and three ill (1), and the file arrives sorted:

row	0	1	2	3	4	5	6	7	8	9	10	11
y	0	0	0	0	0	0	0	0	0	1	1	1

Hold out four rows. How many ill patients land in the test set?

If you take the last four	3 of 3 — and the model trains on <i>no</i> ill patients
If you shuffle	0, 1, 2 or 3, at random
If you stratify	always 1, because $3 \times \frac{4}{12} = 1$

Six seeds, measured

stratify=None gave 2, 1, 1, 0, 1, 1 ill patients in the test set. stratify=y gave 1, 1, 1, 1, 1, 1.

One of those six honest experiments could not compute a recall at all.

Predict first #1

load_wine: 178 rows, 13 features, 3 classes. A decision tree, a 75/25 split. Change nothing but random_state.

```
for seed in range(12):  
    Xtr, Xte, ytr, yte = train_test_split(Xw, yw, test_size=0.25,  
                                          random_state=seed)  
    print(DecisionTreeClassifier(random_state=0).fit(Xtr, ytr).score(Xte, yte))
```

How far apart will the best and the worst seed be?

Predict first #1

load_wine: 178 rows, 13 features, 3 classes. A decision tree, a 75/25 split. Change nothing but random_state.

```
for seed in range(12):  
    Xtr, Xte, ytr, yte = train_test_split(Xw, yw, test_size=0.25,  
                                         random_state=seed)  
    print(DecisionTreeClassifier(random_state=0).fit(Xtr, ytr).score(Xte, yte))
```

How far apart will the best and the worst seed be?

random_state= 0	0.9333	random_state= 6	0.9333
random_state= 1	0.9556	random_state= 7	0.9333
random_state= 2	0.9556	random_state= 8	0.9333
random_state= 3	0.8667	random_state= 9	0.9556
random_state= 4	0.9111	random_state=10	0.9111
random_state= 5	0.9111	random_state=11	0.9556

Predict first #1

load_wine: 178 rows, 13 features, 3 classes. A decision tree, a 75/25 split. Change nothing but random_state.

```
for seed in range(12):  
    Xtr, Xte, ytr, yte = train_test_split(Xw, yw, test_size=0.25,  
                                         random_state=seed)  
    print(DecisionTreeClassifier(random_state=0).fit(Xtr, ytr).score(Xte, yte))
```

How far apart will the best and the worst seed be?

random_state= 0	0.9333	random_state= 6	0.9333
random_state= 1	0.9556	random_state= 7	0.9333
random_state= 2	0.9556	random_state= 8	0.9333
random_state= 3	0.8667	random_state= 9	0.9556
random_state= 4	0.9111	random_state=10	0.9111
random_state= 5	0.9111	random_state=11	0.9556

0.8667 against 0.9556 — **nine points of accuracy, from a number nobody reads.** And these are only the first twelve seeds.

Five hundred seeds

Run the same line 500 times and look at the distribution.

```
wine, decision tree, 500 different random_state values  
mean 0.9132   sd 0.0425   min 0.7556   max 1.0000   range 0.2444  
2.5th-97.5th percentile [0.8106, 0.9778]  
the test set is 44 rows, so one row is worth 0.0227 of accuracy  
5-fold cross-validation instead: 0.9273 +/- 0.0513
```

Five hundred seeds

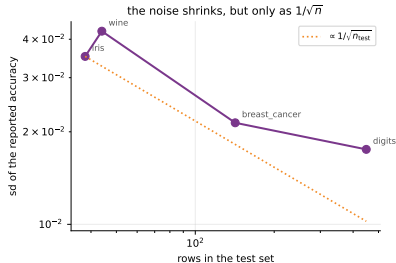
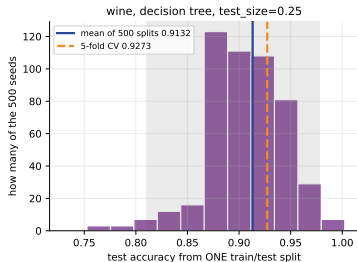
Run the same line 500 times and look at the distribution.

```
wine, decision tree, 500 different random_state values
mean 0.9132   sd 0.0425   min 0.7556   max 1.0000   range 0.2444
2.5th-97.5th percentile [0.8106, 0.9778]
the test set is 44 rows, so one row is worth 0.0227 of accuracy
5-fold cross-validation instead: 0.9273 +/- 0.0513
```

A paper reporting 1.0000 and a paper reporting 0.7556 ran the same code

The 95% interval is [0.81, 0.98], a width of **seventeen points**. With 44 test rows, a single patient moves the headline by 2.3 points. **“We achieved 95.6% accuracy” is not a result. It is one draw from a distribution.**

Drawn: the distribution, and how it shrinks



Left: the 500 seeds, with the shaded band the middle 95%. Right: the same experiment on four datasets. **More test rows means a tighter number — but only as $1/\sqrt{n}$, so to halve the noise you must quadruple the test set**, and every row you move into the test set is a row the model cannot learn from.

It is not just noisy — it ranks models wrongly

An unpruned tree against a max_depth=3 tree, on the same 500 splits of wine.

```
mean difference -0.0021 sd 0.0276
the depth-3 tree won on 119 splits, the unpruned tree on 74, tied on 307
most extreme single-split verdicts: +0.1556 and -0.1333
```


It is not just noisy — it ranks models wrongly

An unpruned tree against a `max_depth=3` tree, on the same 500 splits of wine.

```
mean difference -0.0021  sd 0.0276
the depth-3 tree won on 119 splits, the unpruned tree on 74, tied on 307
most extreme single-split verdicts: +0.1556 and -0.1333
```

Two models that differ by 0.002; one experiment that swings by 0.29

On average these models are indistinguishable. Yet a single split declares a winner 193 times out of 500, and the size of its verdict runs from +0.16 to -0.13. **If your model comparison rests on one split, you have measured the split, not the models.**

It is not just noisy — it ranks models wrongly

An unpruned tree against a `max_depth=3` tree, on the same 500 splits of wine.

```
mean difference -0.0021  sd 0.0276
the depth-3 tree won on 119 splits, the unpruned tree on 74, tied on 307
most extreme single-split verdicts: +0.1556 and -0.1333
```

Two models that differ by 0.002; one experiment that swings by 0.29

On average these models are indistinguishable. Yet a single split declares a winner 193 times out of 500, and the size of its verdict runs from +0.16 to -0.13. **If your model comparison rests on one split, you have measured the split, not the models.**

The fix is not a better seed. It is to **average over many splits** — which is cross-validation, and it is the middle third of this lecture.

The assumption every random split makes

Rows are **independent draws from one population**, so any random subset is as good as any other.

When it fails	Why	What to use
Skewed classes	a fold may contain few or no minority rows	StratifiedKFold, stratify=y
Repeated measurements	rows from the same patient, sensor, image or customer are near-duplicates	GroupKFold, GroupShuffleSplit
Time order	tomorrow is predicted from yesterday, not from next week	TimeSeriesSplit

The test for all three

Could a test row have been produced by something the training set already contains? If yes, your score is measuring recall of the training set.

By hand, then in code: three folds of twelve rows

The same twelve patients, 9 healthy and 3 ill, split three ways.

```
KFold(3, shuffle=False)
  fold 1 test rows [0, 1, 2, 3]   y [0, 0, 0, 0] minority 0
  fold 2 test rows [4, 5, 6, 7]   y [0, 0, 0, 0] minority 0
  fold 3 test rows [8, 9, 10, 11] y [0, 1, 1, 1] minority 3
StratifiedKFold(3)
  fold 1 test rows [0, 1, 2, 9]   y [0, 0, 0, 1] minority 1
  fold 2 test rows [3, 4, 5, 10]  y [0, 0, 0, 1] minority 1
  fold 3 test rows [6, 7, 8, 11]  y [0, 0, 0, 1] minority 1
```

By hand, then in code: three folds of twelve rows

The same twelve patients, 9 healthy and 3 ill, split three ways.

```
KFold(3, shuffle=False)
  fold 1 test rows [0, 1, 2, 3]   y [0, 0, 0, 0] minority 0
  fold 2 test rows [4, 5, 6, 7]   y [0, 0, 0, 0] minority 0
  fold 3 test rows [8, 9, 10, 11] y [0, 1, 1, 1] minority 3
StratifiedKFold(3)
  fold 1 test rows [0, 1, 2, 9]   y [0, 0, 0, 1] minority 1
  fold 2 test rows [3, 4, 5, 10]  y [0, 0, 0, 1] minority 1
  fold 3 test rows [6, 7, 8, 11]  y [0, 0, 0, 1] minority 1
```

Stratification is not clever. It deals the classes out separately. Three ill patients, three folds, one each — exactly the arithmetic you did on paper. KFold without `shuffle` takes contiguous blocks, which on a sorted file is the worst possible choice.

A skewed target, and a fold with nothing to learn from

200 rows, 10 positives (5%), and the file is sorted by label — which is how exports from a database usually arrive.

```
KFold(5, shuffle=False)           positives per fold [10, 0, 0, 0, 0]
KFold(5, shuffle=True, random_state=0) positives per fold [3, 1, 2, 3, 1]
StratifiedKFold(5, shuffle=True, random_state=0) positives per fold [2, 2, 2, 2, 2]
```

A skewed target, and a fold with nothing to learn from

200 rows, 10 positives (5%), and the file is sorted by label — which is how exports from a database usually arrive.

```
KFold(5, shuffle=False)           positives per fold [10, 0, 0, 0, 0]
KFold(5, shuffle=True, random_state=0) positives per fold [3, 1, 2, 3, 1]
StratifiedKFold(5, shuffle=True, random_state=0) positives per fold [2, 2, 2, 2, 2]

KFold(5, shuffle=False) class counts, [negatives positives]:
fold 1  train [160, 0]   test [30, 10]
fold 2  train [150, 10]  test [40,  0]   ... and the same for folds 3-5
```

A skewed target, and a fold with nothing to learn from

200 rows, 10 positives (5%), and the file is sorted by label — which is how exports from a database usually arrive.

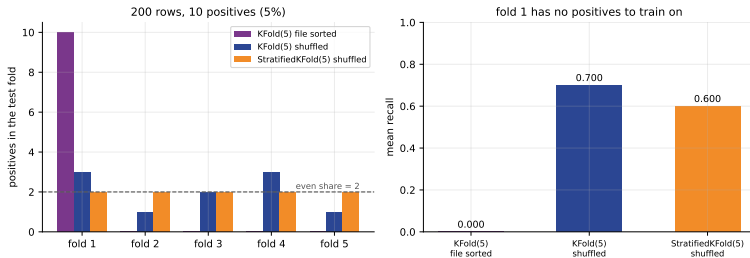
```
KFold(5, shuffle=False)           positives per fold [10, 0, 0, 0, 0]
KFold(5, shuffle=True, random_state=0) positives per fold [3, 1, 2, 3, 1]
StratifiedKFold(5, shuffle=True, random_state=0) positives per fold [2, 2, 2, 2, 2]

KFold(5, shuffle=False) class counts, [negatives positives]:
fold 1  train [160, 0]   test [30, 10]
fold 2  train [150, 10]  test [40,  0]   ... and the same for folds 3-5
```

Fold 1's training set contains *one class*

The classifier cannot be fitted at all, so that fold returns nan. Folds 2–5 have no positives to test on, so each scores accuracy 1.0000. **The reported mean accuracy is 1.0000 and the reported mean recall is 0.0000.**

Drawn: stratification on a 5% minority



Left: positives per test fold. Shuffling fixes the catastrophe but still leaves folds with 1 and folds with 3; stratifying gives 2, 2, 2, 2, 2. Right: the recall each choice reports — 0.000, 0.700, 0.600. **Shuffling alone got a higher number than stratifying, and it got it from folds of unequal difficulty.**

Predict first #2: forty patients, fifteen windows each

40 patients, 15 signal windows each, 600 rows. Each patient's **diagnosis is a coin flip**. Each patient has a random 10-dimensional “fingerprint” that says nothing about the diagnosis; the features are that fingerprint plus noise.

There is nothing to learn. What does a random 5-fold report?

Predict first #2: forty patients, fifteen windows each

40 patients, 15 signal windows each, 600 rows. Each patient's **diagnosis is a coin flip**. Each patient has a random 10-dimensional “fingerprint” that says nothing about the diagnosis; the features are that fingerprint plus noise.

There is nothing to learn. What does a random 5-fold report?

```
random 5-fold, rows shuffled : 1.0000 +/- 0.0000
GroupKFold(5) by patient      : 0.6483 +/- 0.1984
overstatement                  : +0.3517
patients in BOTH train and test, random fold 1 : 37 of 40
patients in BOTH train and test, GroupKFold fold 1: 0
```

Predict first #2: forty patients, fifteen windows each

40 patients, 15 signal windows each, 600 rows. Each patient's **diagnosis is a coin flip**. Each patient has a random 10-dimensional “fingerprint” that says nothing about the diagnosis; the features are that fingerprint plus noise.

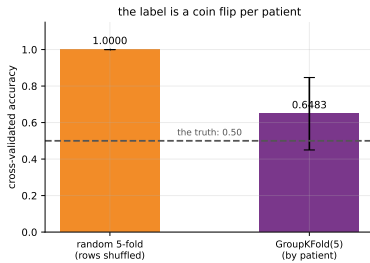
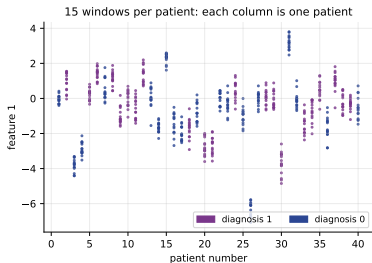
There is nothing to learn. What does a random 5-fold report?

```
random 5-fold, rows shuffled : 1.0000 +/- 0.0000
GroupKFold(5) by patient      : 0.6483 +/- 0.1984
overstatement                  : +0.3517
patients in BOTH train and test, random fold 1 : 37 of 40
patients in BOTH train and test, GroupKFold fold 1: 0
```

1.0000 ± 0.0000 on a coin flip

37 of 40 patients had windows on both sides of fold 1, so the model looked up the patient and read off the answer. **Note the second number too:** ±0.1984. Grouped, the honest estimate has an honest error bar, because you have 40 independent observations and not 600.

Drawn: the rows are not independent



Left: each vertical column is one patient's fifteen windows. They are tight, and they all carry the same label. Right: the two verdicts. **Ask “what is the unit of independence?” before you split. It is the patient, not the window; the customer, not the transaction; the document, not the sentence.**

Time-ordered rows: the most persuasive failure in the le

600 days of a drifting, seasonal series. Features: the day index and two seasonal terms. A random forest.

```
random 75/25 split          R2 +0.9975
train on the first 75% of days R2 -2.7683
TimeSeriesSplit(5), per fold [-1.8789, -0.0513, 0.0816, -4.1669, -8.0681]
TimeSeriesSplit(5), mean    R2 -2.8167
overstatement from shuffling +3.8142
92 of the 150 test days have BOTH neighbours in the training set (61.3%)
```

Time-ordered rows: the most persuasive failure in the le

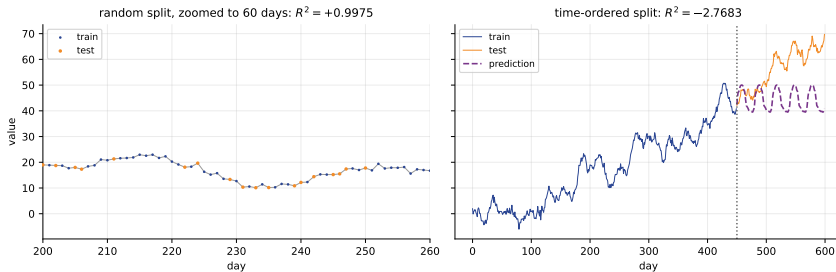
600 days of a drifting, seasonal series. Features: the day index and two seasonal terms. A random forest.

```
random 75/25 split      R2 +0.9975
train on the first 75% of days  R2 -2.7683
TimeSeriesSplit(5), per fold  [-1.8789, -0.0513, 0.0816, -4.1669, -8.0681]
TimeSeriesSplit(5), mean    R2 -2.8167
overstatement from shuffling +3.8142
92 of the 150 test days have BOTH neighbours in the training set (61.3%)
```

$R^2 = +0.9975$, and the honest answer is -2.82

A random split leaves almost every test day *between two training days*, so the model interpolates. In production it must **extrapolate**, and a forest cannot: its predictions are bounded by the training targets. **An R^2 below zero means you would have done better predicting the mean.**

Drawn: interpolation against extrapolation



Left, zoomed to sixty days: orange test points sitting between blue training points. Right: trained on the first 450 days, the forest's prediction **flattens the moment it leaves the training range** while the series keeps climbing.

And the wrong splitter chooses the wrong model

Two candidates on the same 600 days.

A: calendar features + random forest	shuffled 5-fold R2 +0.9974
	TimeSeriesSplit R2 -2.8167
B: lag-1 + trailing mean + linear	shuffled 5-fold R2 +0.9967
	TimeSeriesSplit R2 +0.9230

And the wrong splitter chooses the wrong model

Two candidates on the same 600 days.

A: calendar features + random forest	shuffled 5-fold R2 +0.9974
	TimeSeriesSplit R2 -2.8167
B: lag-1 + trailing mean + linear	shuffled 5-fold R2 +0.9967
	TimeSeriesSplit R2 +0.9230

Shuffled cross-validation ranks A above B, and A is useless

0.9974 against 0.9967: on the shuffled evidence you ship model A. Evaluated in time order, A is -2.82 and B is $+0.92$. **The wrong splitter does not merely inflate a number; it inverts the decision the number was for.**

And the wrong splitter chooses the wrong model

Two candidates on the same 600 days.

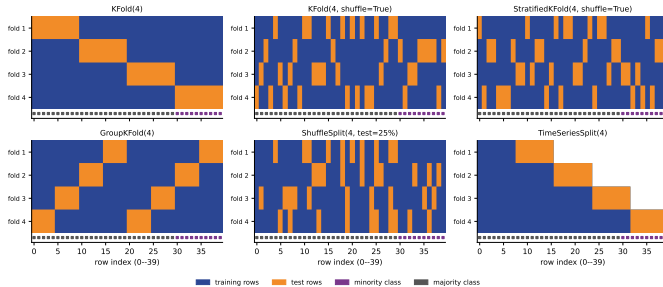
A: calendar features + random forest	shuffled 5-fold R2 +0.9974
	TimeSeriesSplit R2 -2.8167
B: lag-1 + trailing mean + linear	shuffled 5-fold R2 +0.9967
	TimeSeriesSplit R2 +0.9230

Shuffled cross-validation ranks A above B, and A is useless

0.9974 against 0.9967: on the shuffled evidence you ship model A. Evaluated in time order, A is -2.82 and B is $+0.92$. **The wrong splitter does not merely inflate a number; it inverts the decision the number was for.**

Same rule as the grouped case: **if a test row could not have existed at the moment of prediction, it must not be in the test set.**

Drawn: six splitters, from their actual indices



Forty rows, ten of them minority, arriving sorted, in eight groups of five. Minority rows per test fold: KFold [0, 0, 0, 10]; shuffled [0, 6, 2, 2]; stratified [2, 2, 3, 3]; GroupKFold [5, 5, 0, 0]; TimeSeriesSplit [0, 0, 2, 8]. **Every picture here was drawn from the indices the splitter really returned.**

k -fold: use every row for testing, exactly once

The recipe

1. Cut the rows into k folds.
2. For each fold: train on the other $k - 1$, test on this one.
3. Report the mean of the k scores — and the spread.

What it buys

Every row is tested on once, so the estimate uses all n rows rather than $n/4$; and you get k numbers instead of one, so you can see how variable the answer is.

The arithmetic, by hand

With $n = 5000$ rows:

	fits	rows each	row-fits
2-fold	2	2500	5,000
5-fold	5	4000	20,000
10-fold	10	4500	45,000
LOOCV	5000	4999	24,995,000

k -fold costs k times a single fit. That is the whole cost model.

k , measured: bias, and cost

load_breast_cancer, 569 rows, StandardScaler + logistic regression.

2-fold	2 fits	trains on 284 rows	accuracy 0.9859	0.01 s
5-fold	5 fits	trains on 455 rows	accuracy 0.9789	0.01 s
10-fold	10 fits	trains on 512 rows	accuracy 0.9772	0.03 s
20-fold	20 fits	trains on 540 rows	accuracy 0.9790	0.06 s
L00CV	569 fits	trains on 568 rows	accuracy 0.9789	1.62 s
L00CV: 569 fits against 5-fold's 5 = 113.8x as many (the seconds vary)				

k , measured: bias, and cost

load_breast_cancer, 569 rows, StandardScaler + logistic regression.

2-fold	2 fits	trains on 284 rows	accuracy 0.9859	0.01 s
5-fold	5 fits	trains on 455 rows	accuracy 0.9789	0.01 s
10-fold	10 fits	trains on 512 rows	accuracy 0.9772	0.03 s
20-fold	20 fits	trains on 540 rows	accuracy 0.9790	0.06 s
L00CV	569 fits	trains on 568 rows	accuracy 0.9789	1.62 s
L00CV: 569 fits against 5-fold's 5 = 113.8x as many (the seconds vary)				

Read the second column, not the third

2-fold trains on 284 rows and 20-fold on 540. **Small k is pessimistic because it starves the model** — which is the bias of k -fold, and here it is worth about 0.002 because a 569-row learning curve is already flat. On a small or steep dataset it is worth much more.

k , measured: bias, and cost

load_breast_cancer, 569 rows, StandardScaler + logistic regression.

2-fold	2 fits	trains on 284 rows	accuracy 0.9859	0.01 s
5-fold	5 fits	trains on 455 rows	accuracy 0.9789	0.01 s
10-fold	10 fits	trains on 512 rows	accuracy 0.9772	0.03 s
20-fold	20 fits	trains on 540 rows	accuracy 0.9790	0.06 s
L00CV	569 fits	trains on 568 rows	accuracy 0.9789	1.62 s
L00CV: 569 fits against 5-fold's 5 = 113.8x as many (the seconds vary)				

Read the second column, not the third

2-fold trains on 284 rows and 20-fold on 540. **Small k is pessimistic because it starves the model** — which is the bias of k -fold, and here it is worth about 0.002 because a 569-row learning curve is already flat. On a small or steep dataset it is worth much more.

Measured learning curve: 50 rows $\rightarrow$ 0.9566, 200 $\rightarrow$ 0.9709, 540 $\rightarrow$ 0.9776.

Predict first #3: which k gives the steadiest answer?

Repeat the whole cross-validation with 40 different shuffles and look at the spread of **the mean**.

Textbooks say LOOCV has high variance. Does it, here?

Predict first #3: which k gives the steadiest answer?

Repeat the whole cross-validation with 40 different shuffles and look at the spread of **the mean**.

Textbooks say LOOCV has high variance. Does it, here?

2-fold	mean of means	0.9754	sd of the mean	0.0042	range	0.0194
5-fold	mean of means	0.9782	sd of the mean	0.0030	range	0.0123
10-fold	mean of means	0.9786	sd of the mean	0.0025	range	0.0106
20-fold	mean of means	0.9791	sd of the mean	0.0019	range	0.0072
LOOCV	mean	0.9789	sd of the mean	0.0000	(deterministic)	
one 80/20 split, 40 seeds:	mean	0.9772	sd	0.0114	range	0.0526

Predict first #3: which k gives the steadiest answer?

Repeat the whole cross-validation with 40 different shuffles and look at the spread of **the mean**.

Textbooks say LOOCV has high variance. Does it, here?

2-fold	mean of means	0.9754	sd of the mean	0.0042	range	0.0194
5-fold	mean of means	0.9782	sd of the mean	0.0030	range	0.0123
10-fold	mean of means	0.9786	sd of the mean	0.0025	range	0.0106
20-fold	mean of means	0.9791	sd of the mean	0.0019	range	0.0072
LOOCV	mean	0.9789	sd of the mean	0.0000	(deterministic)	
one 80/20 split, 40 seeds:	mean	0.9772	sd	0.0114	range	0.0526

On this dataset the variance *falls* with k , and LOOCV has none

LOOCV does not shuffle, so re-running it changes nothing. **Report the experiment you ran, not the one the textbook predicts:** the high-variance claim for LOOCV is about *correlated folds across different datasets*, and it is not what this measurement shows. What LOOCV certainly is, is expensive: 569 fits against 5-fold's 5, 113.8× as many on any machine. Quote that, not the seconds — the wall-clock ratio wandered between 112 and 131 over four repeats of the same measurement.

Two traps in that table

```
sd ACROSS FOLDS: 5-fold 0.0142  LOOCV 0.1437  <- not comparable
```

Two traps in that table

```
sd ACROSS FOLDS: 5-fold 0.0142  LOOCV 0.1437  <- not comparable
```

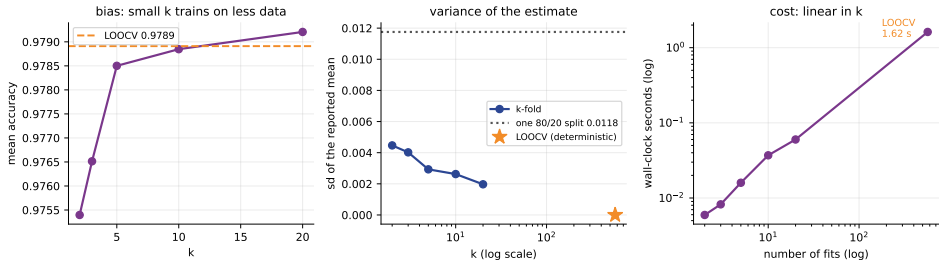
0.1437 is not “LOOCV is unstable”

Each LOOCV fold scores 0 or 1 on a single row, so the across-fold sd is just $\sqrt{p(1-p)}$. **It says nothing about the estimator.** The number that matters is the sd of the *mean*, and that is the previous slide.

And a single split is worse than all of them

One 80/20 split: sd 0.0114, range 0.0526.
Five-fold: sd 0.0030. **Cross-validating is roughly four times steadier here, for five times the compute.**

Drawn: bias, variance and cost against k



Left: the mean rises with k and flattens — that is the bias going away. Middle: the spread of the estimate falls with k ; the dotted line is a single 80/20 split. Right: cost is exactly linear in the number of fits. **Five or ten folds is where these three curves meet.**

More folds, or more repeats?

StratifiedKFold(5)	5 fits	accuracy 0.9789 +/- 0.0142	0.02 s
RepeatedStratifiedKFold(5 x 10)	50 fits	accuracy 0.9772 +/- 0.0147	0.17 s
StratifiedShuffleSplit(50, test 20%)	50 fits	accuracy 0.9767 +/- 0.0122	0.18 s

More folds, or more repeats?

StratifiedKFold(5)	5 fits	accuracy 0.9789 +/- 0.0142	0.02 s
RepeatedStratifiedKFold(5 x 10)	50 fits	accuracy 0.9772 +/- 0.0147	0.17 s
StratifiedShuffleSplit(50, test 20%)	50 fits	accuracy 0.9767 +/- 0.0122	0.18 s

StratifiedKFold(5)

the default. Use it.

RepeatedStratifiedKFold

when n is small and you need the estimate to stop wobbling.

ShuffleSplit

when you want the test size and the number of repeats to be independent of each other.

LeaveOneOut

when n is tiny and a fit is cheap.

GroupKFold / TimeSeriesSplit

whenever the rows are not independent. **Not optional.**

More folds, or more repeats?

StratifiedKFold(5)	5 fits	accuracy 0.9789 +/- 0.0142	0.02 s
RepeatedStratifiedKFold(5 x 10)	50 fits	accuracy 0.9772 +/- 0.0147	0.17 s
StratifiedShuffleSplit(50, test 20%)	50 fits	accuracy 0.9767 +/- 0.0122	0.18 s

StratifiedKFold(5)

the default. Use it.

RepeatedStratifiedKFold

when n is small and you need the estimate to stop wobbling.

ShuffleSplit

when you want the test size and the number of repeats to be independent of each other.

LeaveOneOut

when n is tiny and a fit is cheap.

GroupKFold / TimeSeriesSplit

whenever the rows are not independent. **Not optional.**

Repeating is cheaper than it looks and it is the honest way to say “this difference is real”.

The moment you tune, the score stops being an estimate

Three-way split on breast_cancer: train 341, validation 114, test 114.

```
C= 0.01 validation 0.9561
C= 0.10 validation 0.9561
C= 1.00 validation 0.9474
C= 10.00 validation 0.9474
C=100.00 validation 0.9298
chosen C=0.01 validation 0.9561 TEST 0.9386
```

The moment you tune, the score stops being an estimate

Three-way split on breast_cancer: train 341, validation 114, test 114.

```
C= 0.01 validation 0.9561
C= 0.10 validation 0.9561
C= 1.00 validation 0.9474
C= 10.00 validation 0.9474
C=100.00 validation 0.9298
chosen C=0.01 validation 0.9561 TEST 0.9386
```

0.9561 is a maximum over five tries; 0.9386 is the estimate

You picked the largest of five noisy numbers, so it is biased upward by construction — the same reason the tallest of five random people is taller than average. **The test set is the only number in that block you may quote**, and you may quote it once.

The moment you tune, the score stops being an estimate

Three-way split on breast_cancer: train 341, validation 114, test 114.

```
C= 0.01 validation 0.9561
C= 0.10 validation 0.9561
C= 1.00 validation 0.9474
C= 10.00 validation 0.9474
C=100.00 validation 0.9298
chosen C=0.01 validation 0.9561 TEST 0.9386
```

0.9561 is a *maximum over five tries*; 0.9386 is the estimate

You picked the largest of five noisy numbers, so it is biased upward by construction — the same reason the tallest of five random people is taller than average. **The test set is the only number in that block you may quote**, and you may quote it once.

Cross-validation replaces the validation set; it does not remove the need for a validation step. If the tuning happens inside the same folds you report, the bias comes straight back.

Predict first #4: tuning on pure noise

100 rows, 200 features, everything from $\mathcal{N}(0, 1)$. The labels are **coin flips**. Tune an RBF SVM over 5 values of C and 4 of γ with 5-fold cross-validation, then report the best cross-validated score the search found.

The truth is 0.50. What comes out?

Predict first #4: tuning on pure noise

100 rows, 200 features, everything from $\mathcal{N}(0, 1)$. The labels are **coin flips**. Tune an RBF SVM over 5 values of C and 4 of γ with 5-fold cross-validation, then report the best cross-validated score the search found.

The truth is 0.50. What comes out?

```
tune and report on the SAME folds : 0.5820 +/- 0.0314
nested cross-validation             : 0.5265 +/- 0.0419
optimism                           : +0.0555
the wrong way beat 0.5 on 20 of 20 trials
```

Predict first #4: tuning on pure noise

100 rows, 200 features, everything from $\mathcal{N}(0, 1)$. The labels are **coin flips**. Tune an RBF SVM over 5 values of C and 4 of γ with 5-fold cross-validation, then report the best cross-validated score the search found.

The truth is 0.50. What comes out?

```
tune and report on the SAME folds : 0.5820 +/- 0.0314
nested cross-validation             : 0.5265 +/- 0.0419
optimism                           : +0.0555
the wrong way beat 0.5 on 20 of 20 trials
```

+0.0555 out of thin air, on 20 trials out of 20

`best_score_` is the **maximum** of twenty cross-validated numbers, each with a standard error of a few points. The maximum of twenty noisy numbers is bigger than their mean. `best_score_` is a **selection**, **not an estimate**, and it must never be reported as performance.

And on real data, where there is signal

A 120-row subsample of breast_cancer, the same grid, 20 trials.

```
tune and report on the SAME folds : 0.9529 +/- 0.0054
nested cross-validation             : 0.9317 +/- 0.0094
optimism                           : +0.0212
paired t over 20 trials: t = 9.58, p = 1.1e-08
the wrong way was higher on 19 of 20 trials
```


And on real data, where there is signal

A 120-row subsample of breast_cancer, the same grid, 20 trials.

```
tune and report on the SAME folds : 0.9529 +/- 0.0054
nested cross-validation             : 0.9317 +/- 0.0094
optimism                           : +0.0212
paired t over 20 trials: t = 9.58, p = 1.1e-08
the wrong way was higher on 19 of 20 trials
```

grid size drives it (pure noise, every row should read 0.5000):

1 combinations	same-fold 0.5467	nested 0.5467	optimism +0.0000
4 combinations	same-fold 0.5767	nested 0.5567	optimism +0.0200
20 combinations	same-fold 0.5833	nested 0.5217	optimism +0.0617
60 combinations	same-fold 0.5850	nested 0.5067	optimism +0.0783

And on real data, where there is signal

A 120-row subsample of breast_cancer, the same grid, 20 trials.

```
tune and report on the SAME folds : 0.9529 +/- 0.0054
nested cross-validation             : 0.9317 +/- 0.0094
optimism                           : +0.0212
paired t over 20 trials: t = 9.58, p = 1.1e-08
the wrong way was higher on 19 of 20 trials
```

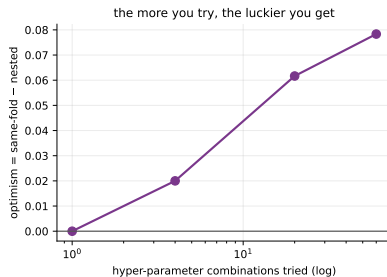
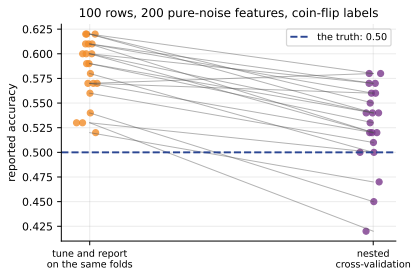
grid size drives it (pure noise, every row should read 0.5000):

1 combinations	same-fold 0.5467	nested 0.5467	optimism +0.0000
4 combinations	same-fold 0.5767	nested 0.5567	optimism +0.0200
20 combinations	same-fold 0.5833	nested 0.5217	optimism +0.0617
60 combinations	same-fold 0.5850	nested 0.5067	optimism +0.0783

With one combination the optimism is *exactly* zero

That row is the proof of the mechanism: with nothing to choose between, there is nothing to over-fit. **The optimism is the price of the search, and it grows with the size of the grid.**

Drawn: twenty trials, paired



Left: each grey line is one trial. Every orange point is above 0.50; the purple points scatter around it as they should. Right: optimism against grid size, from +0.0000 at one combination to +0.0783 at sixty.

Nested cross-validation, in four lines

```
inner = StratifiedKFold(5, shuffle=True, random_state=0)
outer = StratifiedKFold(5, shuffle=True, random_state=0)
search = GridSearchCV(pipe, grid, cv=inner) # chooses the hyper-parameters
score = cross_val_score(search, X, y, cv=outer) # estimates the performance
```

The inner loop **selects**; the outer loop **estimates**. The outer test fold is never involved in choosing anything — not the scaler's mean, not the feature subset, not C , not γ .

Nested cross-validation, in four lines

```
inner = StratifiedKFold(5, shuffle=True, random_state=0)
outer = StratifiedKFold(5, shuffle=True, random_state=0)
search = GridSearchCV(pipe, grid, cv=inner) # chooses the hyper-parameters
score = cross_val_score(search, X, y, cv=outer) # estimates the performance
```

The inner loop **selects**; the outer loop **estimates**. The outer test fold is never involved in choosing anything — not the scaler's mean, not the feature subset, not C , not γ .

Cost, and the honest answer to “is it worth it?”

$5 \times 5 \times 20 = 500$ fits instead of 100. **Use nested CV when you are reporting a number. Use plain GridSearchCV when you are choosing a model and will evaluate it later on a held-out test set.** Both are correct; quoting `best_score_` as your result is not.

By hand: one thousand patients, ten of them ill

A model that always says “healthy”:

	predicted healthy	predicted ill
actually healthy	TN = 990	FP = 0
actually ill	FN = 10	TP = 0

$$\text{accuracy} = \frac{990 + 0}{1000} = \mathbf{0.990}, \quad \text{recall} = \frac{0}{0 + 10} = \mathbf{0}, \quad \text{precision} = \frac{0}{0} \text{ undefined.}$$

99% accurate, and it has never once found an ill patient

Accuracy is $\frac{TP+TN}{TP+TN+FP+FN}$, and when TN is 99% of the table, accuracy is a report on TN.

Recall asks about the row you care about; precision asks about the column you act on.

Predict first #5

5000 rows, 50 positives (1%), twenty features, six of them informative. First a `DummyClassifier` that always predicts the majority; then a real logistic regression.

Which of the two has the higher accuracy?

Predict first #5

5000 rows, 50 positives (1%), twenty features, six of them informative. First a DummyClassifier that always predicts the majority; then a real logistic regression.

Which of the two has the higher accuracy?

```
DummyClassifier(most_frequent) accuracy 0.9900 recall 0.0000 PR-AUC 0.0100
LogisticRegression           accuracy 0.9867 recall 0.0000 PR-AUC 0.1075
    confusion matrix [[1480, 5], [15, 0]]      ROC-AUC 0.8553
accuracy says the model is worth -0.0033 over the dummy
PR-AUC says it is worth          +0.0975
```


Predict first #5

5000 rows, 50 positives (1%), twenty features, six of them informative. First a DummyClassifier that always predicts the majority; then a real logistic regression.

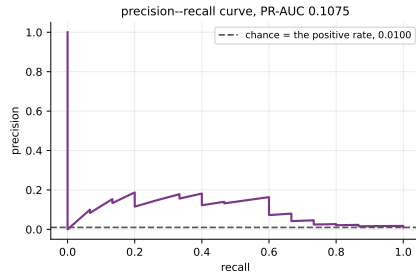
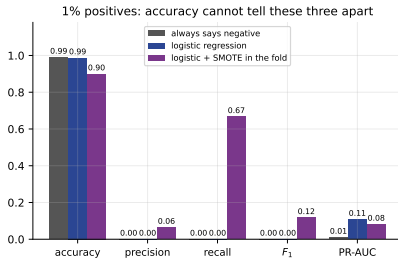
Which of the two has the higher accuracy?

```
DummyClassifier(most_frequent) accuracy 0.9900 recall 0.0000 PR-AUC 0.0100
LogisticRegression            accuracy 0.9867 recall 0.0000 PR-AUC 0.1075
    confusion matrix [[1480, 5], [15, 0]]      ROC-AUC 0.8553
accuracy says the model is worth -0.0033 over the dummy
PR-AUC says it is worth          +0.0975
```

The dummy *wins* on accuracy

The real model has learned a great deal — ROC-AUC 0.8553, PR-AUC nearly eleven times chance — and at the default threshold of 0.5 it still labels almost everything negative, so accuracy *rates it below a model that has learned nothing*. **Two failures at once: the metric is wrong, and the threshold is wrong.**

Drawn: five metrics, three models



Left: accuracy is 0.99, 0.99, 0.90 for a useless model, a good model and a good model with the threshold moved. **The one metric that is the same for all three is the one everybody quotes.** Right: the precision–recall curve, whose baseline is the positive rate 0.01, not 0.5.

Four things you can do about it

	What it does	What it costs
Random oversampling	copies minority rows until the classes balance	duplicates: the model can memorise them
Random undersampling	throws majority rows away until the classes balance	discards most of your data
SMOTE	invents minority rows between existing ones	interpolates into regions where no data was observed
Class weights	multiplies the loss of minority errors, no rows added or removed	none, and it is one keyword argument

And the one that is not on the list

Move the threshold. A classifier gives you a probability; 0.5 is a convention, not a result. Choosing the threshold from the precision–recall curve costs nothing and is often all you need.

By hand: SMOTE is one line of arithmetic

Two minority patients, $A = (1, 1)$ and $B = (3, 5)$. SMOTE draws $u \sim \mathcal{U}(0, 1)$ and returns $A + u(B - A)$:

$u = 0.00$	$\rightarrow$	$[1.0, 1.0]$	(that is A)
$u = 0.25$	$\rightarrow$	$[1.5, 2.0]$	
$u = 0.50$	$\rightarrow$	$[2.0, 3.0]$	(the midpoint)
$u = 1.00$	$\rightarrow$	$[3.0, 5.0]$	(that is B)

By hand: SMOTE is one line of arithmetic

Two minority patients, $A = (1, 1)$ and $B = (3, 5)$. SMOTE draws $u \sim \mathcal{U}(0, 1)$ and returns $A + u(B - A)$:

$u = 0.00$	$\rightarrow$	$[1.0, 1.0]$	(that is A)
$u = 0.25$	$\rightarrow$	$[1.5, 2.0]$	
$u = 0.50$	$\rightarrow$	$[2.0, 3.0]$	(the midpoint)
$u = 1.00$	$\rightarrow$	$[3.0, 5.0]$	(that is B)

```
nn = NearestNeighbors(n_neighbors=6).fit(Xmin) # k = 5 minority neighbours
_, ind = nn.kneighbors(Xmin); ind = ind[:, 1:] # drop the point itself
new = Xmin[base] + u * (Xmin[nbr] - Xmin[base]) # a point on the segment
```

By hand: SMOTE is one line of arithmetic

Two minority patients, $A = (1, 1)$ and $B = (3, 5)$. SMOTE draws $u \sim \mathcal{U}(0, 1)$ and returns $A + u(B - A)$:

$u = 0.00$	$\rightarrow$	$[1.0, 1.0]$	(that is A)
$u = 0.25$	$\rightarrow$	$[1.5, 2.0]$	
$u = 0.50$	$\rightarrow$	$[2.0, 3.0]$	(the midpoint)
$u = 1.00$	$\rightarrow$	$[3.0, 5.0]$	(that is B)

```
nn = NearestNeighbors(n_neighbors=6).fit(Xmin) # k = 5 minority neighbours
_, ind = nn.kneighbors(Xmin); ind = ind[:, 1:] # drop the point itself
new = Xmin[base] + u * (Xmin[nbr] - Xmin[base]) # a point on the segment
```

That is the whole of Chawla et al. (2002). **It is not generating data; it is asserting that the straight line between two ill patients is full of ill patients.** Sometimes true. Sometimes exactly wrong.

The three resamplers, on the 1% problem

```
random oversample  train 3500 -> 6930 rows, class counts [3465, 3465]
random undersample train 3500 ->   70 rows, class counts [35, 35]
SMOTE              train 3500 -> 6930 rows, class counts [3465, 3465]
    3430 of the 3465 minority rows are synthetic (99.0%)
    distinct minority rows: SMOTE 3465, oversampling 35
```

The three resamplers, on the 1% problem

```
random oversample  train 3500 -> 6930 rows, class counts [3465, 3465]
random undersample train 3500 ->   70 rows, class counts [35, 35]
SMOTE              train 3500 -> 6930 rows, class counts [3465, 3465]
                    3430 of the 3465 minority rows are synthetic (99.0%)
                    distinct minority rows: SMOTE 3465, oversampling 35
```

Read the last line twice

Oversampling turns 35 patients into 3465 rows that are copies of 35 **distinct patients**. SMOTE turns them into 3465 distinct points — but every one of them lies on a segment between two of the same 35. **Neither method has added a single new observation.**

The three resamplers, on the 1% problem

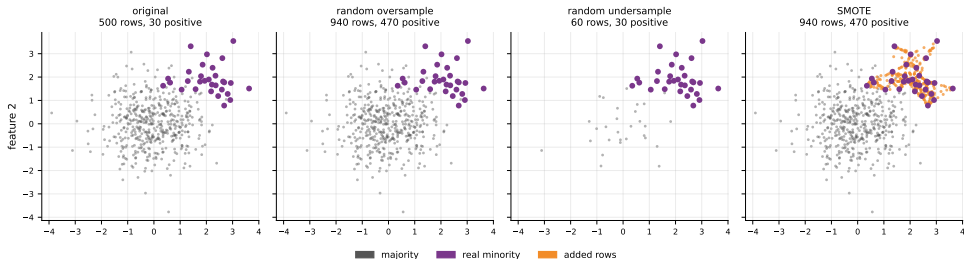
```
random oversample  train  3500 -> 6930 rows, class counts [3465, 3465]
random undersample train  3500 ->   70 rows, class counts [35, 35]
SMOTE              train  3500 -> 6930 rows, class counts [3465, 3465]
                    3430 of the 3465 minority rows are synthetic (99.0%)
                    distinct minority rows: SMOTE 3465, oversampling 35
```

Read the last line twice

Oversampling turns 35 patients into 3465 rows that are copies of 35 **distinct patients**. SMOTE turns them into 3465 distinct points — but every one of them lies on a segment between two of the same 35. **Neither method has added a single new observation.**

And undersampling kept 70 of 3500 rows: 99.0% **of the negatives were thrown away.**

Drawn: what each resampler does to the picture



Purple is real minority, orange is added. Oversampling adds nothing you can see — the new points are *on top of* the old ones. SMOTE's additions trace the segments between neighbours, which is why the cloud looks like a web. Undersampling simply deletes most of the grey.

Does any of it help? Measure, and report what you find

Five-fold, resampled **inside** each fold, logistic regression.

method	accuracy	precision	recall	F1	PR-AUC
always negative	0.9900	0.0000	0.0000	0.0000	0.0100
no treatment	0.9890	0.1333	0.0400	0.0583	0.1916
random oversample	0.8542	0.0517	0.7800	0.0969	0.1369
random undersample	0.7944	0.0390	0.8200	0.0744	0.1142
SMOTE	0.8814	0.0612	0.7600	0.1133	0.1404
class_weight balanced	0.8556	0.0520	0.7800	0.0974	0.1368

Does any of it help? Measure, and report what you find

Five-fold, resampled **inside** each fold, logistic regression.

method	accuracy	precision	recall	F1	PR-AUC
always negative	0.9900	0.0000	0.0000	0.0000	0.0100
no treatment	0.9890	0.1333	0.0400	0.0583	0.1916
random oversample	0.8542	0.0517	0.7800	0.0969	0.1369
random undersample	0.7944	0.0390	0.8200	0.0744	0.1142
SMOTE	0.8814	0.0612	0.7600	0.1133	0.1404
class_weight balanced	0.8556	0.0520	0.7800	0.0974	0.1368

Recall 0.04 $\rightarrow$ 0.78; PR-AUC 0.19 $\rightarrow$ 0.14

Every treatment transforms recall and none of them improves the *ranking*: PR-AUC is highest with no treatment at all. **Resampling moves the decision threshold. It does not create information.** That is a genuine result and it should be reported, not hidden.

Does any of it help? Measure, and report what you find

Five-fold, resampled **inside** each fold, logistic regression.

method	accuracy	precision	recall	F1	PR-AUC
always negative	0.9900	0.0000	0.0000	0.0000	0.0100
no treatment	0.9890	0.1333	0.0400	0.0583	0.1916
random oversample	0.8542	0.0517	0.7800	0.0969	0.1369
random undersample	0.7944	0.0390	0.8200	0.0744	0.1142
SMOTE	0.8814	0.0612	0.7600	0.1133	0.1404
class_weight balanced	0.8556	0.0520	0.7800	0.0974	0.1368

Recall 0.04 $\rightarrow$ 0.78; PR-AUC 0.19 $\rightarrow$ 0.14

Every treatment transforms recall and none of them improves the *ranking*: PR-AUC is highest with no treatment at all. **Resampling moves the decision threshold. It does not create information.** That is a genuine result and it should be reported, not hidden.

Note also that `class_weight="balanced"` matches random oversampling to three decimal places — as it should, since weighting a row by w and copying it w times are the same thing to the loss function.

Predict first #6: one line in the wrong place

The same SMOTE, the same model, the same five folds. The only difference: X , $y = \text{smote}(X, y)$ is called *before* `cross_val_score` instead of inside the loop.

How much does that move F_1 ?

Predict first #6: one line in the wrong place

The same SMOTE, the same model, the same five folds. The only difference: X , y = `smote(X, y)` is called *before* `cross_val_score` instead of inside the loop.

How much does that move F_1 ?

random oversample	BEFORE the split :	F1 0.8607	recall 0.8655	PR-AUC 0.9176
random oversample	INSIDE each fold :	F1 0.0969	recall 0.7800	PR-AUC 0.1369
SMOTE	BEFORE the split :	F1 0.8942	recall 0.9067	PR-AUC 0.9207
SMOTE	INSIDE each fold :	F1 0.1133	recall 0.7600	PR-AUC 0.1404
overstatement in F1: +0.7638 (oversample) +0.7809 (SMOTE)				

Predict first #6: one line in the wrong place

The same SMOTE, the same model, the same five folds. The only difference: X , $y = \text{smote}(X, y)$ is called *before* `cross_val_score` instead of inside the loop.

How much does that move F_1 ?

```
random oversample BEFORE the split : F1 0.8607 recall 0.8655 PR-AUC 0.9176
random oversample INSIDE each fold : F1 0.0969 recall 0.7800 PR-AUC 0.1369
SMOTE             BEFORE the split : F1 0.8942 recall 0.9067 PR-AUC 0.9207
SMOTE             INSIDE each fold : F1 0.1133 recall 0.7600 PR-AUC 0.1404
overstatement in F1: +0.7638 (oversample) +0.7809 (SMOTE)
```

0.894 against 0.113

An eight-fold overstatement, from moving one line. This is by a wide margin the most common mistake in student projects on imbalanced data, and it produces exactly the sort of number that makes a report look finished.

Why: the same row is on both sides

after oversampling before the split, fold 1 has 1980 test rows
of which 990 (50.0%) are byte-identical to a training row
among the 990 MINORITY test rows, 990 (100.0%) are copies

Why: the same row is on both sides

after oversampling before the split, fold 1 has 1980 test rows
of which 990 (50.0%) are byte-identical to a training row
among the 990 MINORITY test rows, 990 (100.0%) are copies

Every single minority row in the test fold is a copy of a row the model trained on. The model is not being tested; it is being asked to remember. SMOTE is only slightly less blatant: its synthetic points are built *from* the training points, so they carry the same information across the boundary.

Why: the same row is on both sides

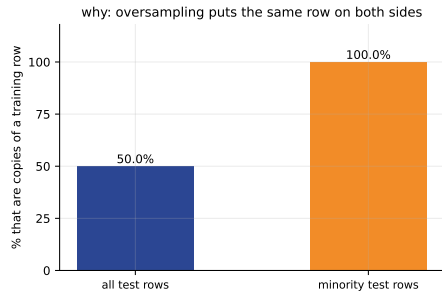
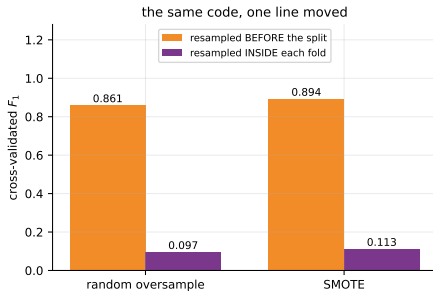
after oversampling before the split, fold 1 has 1980 test rows
of which 990 (50.0%) are byte-identical to a training row
among the 990 MINORITY test rows, 990 (100.0%) are copies

Every single minority row in the test fold is a copy of a row the model trained on. The model is not being tested; it is being asked to remember. SMOTE is only slightly less blatant: its synthetic points are built *from* the training points, so they carry the same information across the boundary.

The rule, stated once

Resampling is part of the model, so it happens inside the fold, on the training part only. The test fold keeps its natural class balance — because that is the balance you will meet in production.

Drawn: the same code, one line moved



Left: F_1 for both resamplers, before and inside. Right: the mechanism — half of every test fold, and **all** of its minority half, are duplicates of training rows.

How bad does imbalance have to get?

The same generator, the minority fraction swept from 50% to 0.2%.

minority 0.9063	50.0% (2500 rows)	always-negative	0.5000	accuracy	0.8586	recall	0.8704	PR-AUC
minority 0.7532	20.0% (1000 rows)	always-negative	0.8000	accuracy	0.8874	recall	0.6210	PR-AUC
minority 0.6098	10.0% (500 rows)	always-negative	0.9000	accuracy	0.9260	recall	0.4400	PR-AUC
minority 0.4392	5.0% (250 rows)	always-negative	0.9500	accuracy	0.9544	recall	0.2720	PR-AUC
minority 0.1916	1.0% (50 rows)	always-negative	0.9900	accuracy	0.9890	recall	0.0400	PR-AUC
minority 0.0277	0.2% (10 rows)	always-negative	0.9980	accuracy	0.9980	recall	0.0000	PR-AUC

How bad does imbalance have to get?

The same generator, the minority fraction swept from 50% to 0.2%.

minority 0.9063	50.0% (2500 rows)	always-negative	0.5000	accuracy	0.8586	recall	0.8704	PR-AUC
minority 0.7532	20.0% (1000 rows)	always-negative	0.8000	accuracy	0.8874	recall	0.6210	PR-AUC
minority 0.6098	10.0% (500 rows)	always-negative	0.9000	accuracy	0.9260	recall	0.4400	PR-AUC
minority 0.4392	5.0% (250 rows)	always-negative	0.9500	accuracy	0.9544	recall	0.2720	PR-AUC
minority 0.1916	1.0% (50 rows)	always-negative	0.9900	accuracy	0.9890	recall	0.0400	PR-AUC
minority 0.0277	0.2% (10 rows)	always-negative	0.9980	accuracy	0.9980	recall	0.0000	PR-AUC

Two columns moving in opposite directions

As the problem gets harder, **accuracy goes up** — 0.86 at balance, 0.998 at 0.2% — while recall falls to zero. **Reported accuracy is a measure of how imbalanced your data is.** There is no threshold at which imbalance “starts”; it is a slope.

The whole thing in one place

```
rng = np.random.default_rng(0) # the one seed for this lecture
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y,
                                     random_state=0)
for C in grid: # tune on the TRAINING part only
    for tr, va in StratifiedKFold(5, shuffle=True,
                                  random_state=0).split(Xtr, ytr):
        Xa, ya = smote(Xtr[tr], ytr[tr], rng) # resample INSIDE the fold
        ...score on Xtr[va], which was never resampled...
final = fit(smote(Xtr, ytr, rng), best_C) # refit on all the training rows
report(final, Xte, yte) # the test set, once
```

```
chosen C=0.01 (inner F1 0.1271)
held-out TEST: accuracy 0.8940 precision 0.0610 recall 0.6667 F1 0.1117 PR-AUC 0.0842
confusion matrix [[1331, 154], [5, 10]]
```

The whole thing in one place

```
rng = np.random.default_rng(0) # the one seed for this lecture
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y,
                                     random_state=0)
for C in grid: # tune on the TRAINING part only
    for tr, va in StratifiedKFold(5, shuffle=True,
                                  random_state=0).split(Xtr, ytr):
        Xa, ya = smote(Xtr[tr], ytr[tr], rng) # resample INSIDE the fold
        ...score on Xtr[va], which was never resampled...
final = fit(smote(Xtr, ytr, rng), best_C) # refit on all the training rows
report(final, Xte, yte) # the test set, once
```

```
chosen C=0.01 (inner F1 0.1271)
held-out TEST: accuracy 0.8940 precision 0.0610 recall 0.6667 F1 0.1117 PR-AUC 0.0842
confusion matrix [[1331, 154], [5, 10]]
```

Ten of the fifteen ill patients found, at the cost of 154 false alarms. That is the real trade-off, stated in patients rather than in decimals — and it is a clinical decision, not a modelling one.

Summary — twelve lines

1. A single train/test score is a random variable. On wine, 500 seeds gave 0.7556 to 1.0000, a 95% interval 17 points wide.
2. That noise reverses model comparisons: two trees differing by 0.002 on average produced single-split verdicts from +0.16 to -0.13.
3. Test-set noise falls only as $1/\sqrt{n_{\text{test}}}$, and every test row is a row you did not train on.
4. Stratify. On a sorted 5% file, KFold put all ten positives in fold 1, left fold 1's *training set with one class*, and reported accuracy 1.0000 with recall 0.0000.
5. Group your rows. 40 patients with coin-flip labels scored 1.0000 ± 0.0000 under a random split and 0.6483 ± 0.1984 under GroupKFold.
6. Respect time. A random split scored $R^2 = +0.9975$; the honest time-ordered answer was -2.8167 — and shuffled CV ranked the useless model *above* the good one.
7. k -fold costs k fits. On 569 rows, LOOCV took about 100×5 -fold to reach the same 0.9789.
8. Small k is pessimistic because it starves the model: 2-fold trains on 284 rows, 20-fold on 540.
9. GridSearchCV.best_score_ is a maximum, not an estimate. On pure noise it read 0.5820 against a nested 0.5265; with one combination in the grid the gap was **exactly zero**.
10. At 1% positives, always-say-no scores 0.9900 and a real model scores 0.9867. **Accuracy ranked the useless model higher.**
11. Oversampling, undersampling, SMOTE and class weights all took recall from 0.04 to about 0.78 — and **none improved PR-AUC**. They move the threshold; they do not create information.
12. **Resampling before the split reported $F_1 = 0.894$ against an honest 0.113**, because 100% of the minority test rows were copies of training rows.

Before the next lecture

Read

notes.pdf — the fold arithmetic worked by hand, the derivation of why `best_score_` is biased, and **10 practice questions with worked solutions**.

Do

Questions 2, 5 and 9. Question 5 is a confusion matrix by hand; question 9 is a debugging story you will meet for real.

This closes Unit IV

Cleaning, features, splitting. Everything after this point assumes you can produce a number you would defend.

One habit to start now

Never report a bare score. Report **the estimator, the splitter, and the spread** — “ 0.9789 ± 0.0142 , stratified 5-fold” — and say what the test set was protected from.

Materials: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Han, Kamber and Pei, *Data Mining: Concepts and Techniques* (3e), Ch. 3.; Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 3–4.; James, Witten, Hastie, Tibshirani and Taylor, *An Introduction to Statistical Learning with Python*, Ch. 5–6, 12.; Scikit-learn documentation (preprocessing, model selection), accessed 2026-08-04.